

UNIVERSIDAD PANAMERICANA

Ingeniería

Ingeniería en sistemas y tecnología de la información y comunicación

Programación de Microprocesadores



Proyecto Final

(Robot Sumo)

Kevin Jósue Leal García (000149700)

Yeimy Melisa Coy Torres (000149948)

Jonathan Enrique Solís Reyes (000149628)

Manuel Eduardo Romero Tesucun (Catedrático)

Petén, junio, 2026

Proyecto Final

(Robot Sumo)

Kevin Jósue Leal García (000149700)

Yeimy Melisa Coy Torres (000149948)

Jonathan Enrique Solís Reyes (000149628)

Manuel Eduardo Romero Tesucun (Catedrático)

Petén, junio, 2026

Introducción

En la primera etapa del proyecto se desarrolló un robot sumo capaz de moverse de forma autónoma a partir de la detección de figuras geométricas y colores mediante visión artificial. Si bien ese enfoque permitió comprobar la integración entre procesamiento de imágenes, comunicación inalámbrica y control de motores, el funcionamiento autónomo no resulta adecuado para una competencia de sumo real, donde se requieren reacciones inmediatas, maniobras de ataque y defensa, y un control total sobre la velocidad y la dirección del robot en cada instante.

Por esta razón, en esta segunda etapa el robot fue transformado para ser operado de manera manual mediante un mando inalámbrico PS4 (DualShock 4) conectado directamente a la ESP32-CAM por Bluetooth. Este cambio elimina la dependencia de la computadora externa, la cámara IP y la red WiFi utilizadas en la primera versión, y traslada el control completo del robot a las manos del operador, permitiendo movimientos más rápidos, embestidas, giros evasivos y frenado activo según las necesidades del combate.

Adicionalmente, se incorporó un mecanismo mecánico de combate de tipo “anti-flipper” (paleta), accionado mediante un servomotor DS3218MG, que permite levantar bruscamente una paleta frontal para intentar voltear o desestabilizar al robot contrario. Este mecanismo se suma al sistema de tracción basado en dos motores JGA25-370 controlados mediante controladores BTS7960, reutilizados de la primera versión del proyecto.

El presente reporte describe la nueva arquitectura de control implementada, el funcionamiento detallado del firmware según el código fuente desarrollado, y los problemas técnicos que surgieron durante la integración del mando PS4, el servomotor anti-flipper y el rediseño mecánico del robot, junto con las soluciones aplicadas para resolverlos.

Objetivos

Objetivo General

Transformar el robot sumo desarrollado en la primera etapa del proyecto, sustituyendo su control autónomo basado en visión artificial por un control manual mediante un mando PS4 conectado por Bluetooth, e incorporar un mecanismo anti-flipper accionado por servomotor para permitir maniobras ofensivas y defensivas durante el combate.

Objetivos Específicos

- Establecer comunicación Bluetooth entre un mando PS4 (DualShock 4) y la ESP32-CAM mediante la librería PS4Controller.
- Configurar el emparejamiento del mando utilizando su dirección MAC para garantizar una conexión estable y exclusiva.
- Implementar un mezclador de joystick tipo “arcade” que combine de forma proporcional el avance y el giro a partir de un único joystick.
- Incorporar rampas de aceleración y un sistema de frenado activo para reducir el patinaje de las llantas y la inercia generada por los motores JGA25-370.
- Integrar un servomotor DS3218MG como mecanismo anti-flipper, controlado mediante señales PWM de 50 Hz.

Robot Sumo: Control Manual Mediante Mando PS4

Arquitectura del Sistema

En esta segunda etapa la arquitectura del sistema se simplificó respecto a la versión basada en visión artificial, ya que la computadora externa, la cámara IP y la comunicación WiFi dejaron de ser necesarias para el control del robot. El nuevo sistema está compuesto por dos capas funcionales que trabajan en conjunto:

Mando PS4 (DualShock 4)

Su función es capturar las acciones del operador mediante el joystick izquierdo y los botones, y transmitirlos de forma inalámbrica mediante Bluetooth hacia la ESP32-CAM.

Sus funciones son:

- Lectura continua de los ejes del joystick izquierdo.
- Lectura del estado de los botones (R1, L2, Círculo, X, Cuadrado).
- Transmisión del estado del mando mediante Bluetooth.

ESP32-CAM (Control)

Su función es recibir el estado del mando mediante la librería PS4Controller, interpretar los comandos y ejecutar las acciones correspondientes sobre los motores y el servomotor anti-flipper.

Sus funciones son:

- Recepción de datos del mando vía Bluetooth.
- Interpretación de botones y ejes del joystick.
- Generación de señales PWM para los motores mediante los BTS7960.
- Control del servomotor anti-flipper.
- Aplicación de rampas de aceleración y frenado activo.

Componentes de Hardware

El sistema físico reutiliza la mayor parte de los componentes de la primera etapa, e incorpora nuevos elementos para el control manual y el mecanismo anti-flipper:

Mando PS4 (DualShock 4)

El DualShock 4 es el control inalámbrico oficial de la consola PlayStation 4, que incorpora conectividad Bluetooth, dos joysticks analógicos, botones digitales (Cuadrado, Círculo, Triángulo, X) y gatillos analógicos (L1, L2, R1, R2). En este proyecto se utilizó para sustituir al sistema de visión artificial como método de generación de comandos, permitiendo un control manual y directo del robot.

Funciones principales:

- Lectura de movimiento mediante el joystick izquierdo.
- Activación de maniobras especiales mediante botones.
- Comunicación Bluetooth con la ESP32-CAM.

ESP32-CAM AI Thinker

La ESP32-CAM se reutilizó de la primera etapa del proyecto, pero en esta versión su función cambió: en lugar de actuar como servidor WiFi receptor de comandos HTTP, opera como dispositivo Bluetooth que recibe directamente el estado del mando PS4 mediante la librería PS4Controller, y se encarga del control completo de los motores y del nuevo mecanismo anti-flipper.

Funciones principales:

- Comunicación Bluetooth con el mando PS4.
- Generación de señales PWM para motores y servomotor.
- Ejecución de la lógica de control en tiempo real.
- Aplicación de rampas de aceleración y frenado activo.

Controlador BTS7960

El BTS7960 es un controlador de potencia basado en un puente H de alta corriente, reutilizado de la primera etapa del proyecto sin modificaciones en su conexionado. Continúa encargándose de regular la velocidad y dirección de los motores mediante las señales PWM generadas por la ESP32, ahora a partir de los comandos recibidos del mando PS4 en lugar de comandos HTTP.

Funciones principales:

- Control de velocidad y dirección de los motores.
- Manejo de corrientes elevadas.
- Implementación de frenado activo.

Motores JGA25-370 (170 RPM, 12V)

Los motores JGA25-370 también se reutilizaron de la primera etapa, pero en esta versión se reajustaron sus parámetros de velocidad y aceleración para lograr un comportamiento más ágil y competitivo: se incrementó la velocidad base, la velocidad máxima de turbo y la duración de la embestida respecto a la configuración inicial, manteniendo activas las rampas de aceleración para evitar el patinaje de las llantas.

Funciones principales:

- Movimiento y tracción del robot.

- Ejecución de giros y embestidas.
- Frenado activo mediante pulso inverso.

Servomotor DS3218MG (Anti-Flipper)

El DS3218MG es un servomotor digital de alto torque, capaz de operar entre 0° y 180°, alimentado con un rango de voltaje de 5V a 8.4V. En este proyecto se utilizó como actuador del mecanismo anti-flipper, una paleta mecánica que el robot puede levantar bruscamente para intentar voltear o desestabilizar al robot contrario durante el combate.

Funciones principales:

- Apertura rápida de la paleta anti-flipper.
- Cierre automático tras un breve tiempo de exposición.
- Generación de fuerza mecánica para maniobras ofensivas.

Fuente de Alimentación

El sistema de alimentación reutiliza las baterías de la primera etapa, pero fue necesario revisar y redistribuir la capacidad de corriente entre los motores, la lógica de control y el nuevo servomotor anti-flipper, debido a los problemas de caída de voltaje detectados durante las pruebas (descritos en la sección de problemas encontrados).

Funciones principales:

- Alimentación de motores, ESP32-CAM y servomotor.
- Suministro de corriente de arranque para los motores JGA25-370.
- Estabilidad eléctrica del sistema durante el combate.

Chasis y Armazón del Robot

Para esta segunda etapa se incorporó un armazón de protección sobre la estructura del robot, lo que requirió reorganizar la distribución física de los componentes electrónicos (motores, controladores BTS7960, ESP32-CAM, batería y servomotor) para liberar espacio de montaje sin afectar el cableado ni el centro de gravedad del robot.

Funciones principales:

- Soporte estructural y protección contra impactos durante el combate.
- Distribución del peso del robot.
- Punto de anclaje para el mecanismo anti-flipper.

Componentes de Software

Librería PS4Controller

Librería para ESP32 que permite establecer una conexión Bluetooth con un mando PS4 (DualShock 4) y leer en tiempo real el estado de sus joysticks y botones. Esta librería sustituyó por completo al servidor HTTP y a la conexión WiFi utilizados en la primera etapa del proyecto.

Funciones principales:

- Emparejamiento Bluetooth mediante dirección MAC.
- Lectura de los ejes analógicos del joystick izquierdo.
- Lectura del estado de botones digitales y gatillos.

Arduino IDE y Núcleo (Core) de ESP32

Entorno de desarrollo y conjunto de librerías base utilizados para programar la ESP32-CAM. La compatibilidad entre la librería PS4Controller y la versión del núcleo de ESP32 instalado fue un factor crítico durante la integración, tal como se describe en la sección de problemas encontrados.

Funciones principales:

- Compilación y carga del firmware.
- Gestión de librerías y dependencias.
- Acceso a las funciones de bajo nivel del ESP32 (LEDC, Bluetooth).

LEDC (Controlador PWM del ESP32)

Periférico interno del ESP32 utilizado para generar las señales PWM que controlan tanto los motores (mediante los BTS7960) como el servomotor anti-flipper. Cada actuador utiliza un canal LEDC independiente, configurado con la frecuencia y resolución adecuadas según el dispositivo que controla.

Funciones principales:

- Generación de PWM a 1 kHz para los motores.
- Generación de PWM a 50 Hz para el servomotor.
- Control independiente de cada actuador.

Distribución de Pines del ESP32-CAM

A diferencia de la primera etapa, en esta versión los pines anteriormente utilizados para el sensor MPU6050 quedaron disponibles, ya que el control de orientación mediante giroscopio no se utiliza en el modo de control manual. La distribución de pines correspondiente al control de motores y al servomotor anti-flipper es la siguiente:

Función	GPIO / Conexión
RPWM Motor Izquierdo	GPIO 13
LPWM Motor Izquierdo	GPIO 12

RPWM Motor Derecho	GPIO 2
LPWM Motor Derecho	GPIO 4
Señal Servo DS3218MG (Anti-Flipper)	GPIO 14
R_EN y L_EN (ambos BTS7960)	5V fijo

El GPIO 14, utilizado previamente para la comunicación I2C con el MPU6050, fue reasignado a la señal de control del servomotor anti-flipper, mientras que el GPIO 15 permanece libre para futuras ampliaciones del sistema.

Mapeo de Control del Mando PS4

El joystick izquierdo concentra todo el movimiento de traslación del robot mediante un mezclador tipo arcade, mientras que los botones del mando se reservan para maniobras especiales de combate. Dentro del bucle principal del programa, las acciones se evalúan con la siguiente prioridad: frenado de emergencia (X), embestida (R1), giro evasivo (Círculo) y, finalmente, el movimiento normal mediante el joystick; de esta manera, las maniobras críticas de combate interrumpen de inmediato cualquier otro movimiento en curso.

Control	Acción
Joystick izquierdo	Movimiento: avanzar, retroceder, girar sobre el eje y curvas proporcionales según la inclinación.
R1	Embestida: rampa rápida hasta la velocidad máxima, seguida de frenado activo.
L2	Turbo: incrementa la velocidad máxima de avance y giro.
Círculo	Giro evasivo de 180° sobre el propio eje del robot.
X (Cross)	Frenado activo de emergencia.
Cuadrado	Activa la secuencia automática del flipper (apertura y cierre).

Explicación Detallada del Funcionamiento Según el Código

Conexión y Emparejamiento Bluetooth

El programa inicia la comunicación Bluetooth mediante la función PS4.begin(), a la cual se le proporciona la dirección MAC del mando PS4 previamente emparejado. Esta dirección fija evita que la ESP32 acepte conexiones de otros mandos cercanos y permite reconectar automáticamente con el mismo control en cada encendido. Mientras el mando no esté conectado, el robot permanece detenido como medida de seguridad.

Mezclador de Joystick (mixJoystick)

El movimiento del robot se calcula a partir de los dos ejes del joystick izquierdo mediante una función de mezcla tipo arcade. El eje vertical determina el avance o retroceso, y el eje horizontal determina el giro; ambos valores se combinan sumándolos para el motor izquierdo y restándolos para el motor derecho, lo que permite generar curvas proporcionales sin necesidad de calcular

ángulos. Antes de aplicar la mezcla se descartan los valores pequeños del joystick (zona muerta) para evitar desplazamientos involuntarios por el ruido del sensor analógico, y al final se aplica un recorte proporcional que conserva la dirección del movimiento, aunque la suma de ambos ejes supere el valor máximo permitido.

Rampas de Aceleración (rampa)

Para evitar que las llantas patinen al iniciar el, la velocidad no se aplica de forma instantánea, sino que se incrementa o disminuye gradualmente en cada ciclo del programa hasta alcanzar el valor solicitado por el joystick. Esta misma lógica de rampa se reutiliza para suavizar el movimiento del servomotor anti-flipper entre su posición de reposo y su posición de ataque.

Frenado Activo (frenoActivo)

Debido a la inercia generada por los motores JGA25-370, el robot continuaba desplazándose brevemente después de soltar el joystick o de finalizar una maniobra. Para contrarrestar este efecto se implementó un frenado activo que aplica un pulso breve en sentido contrario al movimiento antes de detener completamente los motores; esta función se utiliza tanto en el frenado de emergencia (botón X) como al finalizar la embestida (R1) y el giro evasivo (Círculo).

Maniobras Especiales (R1 y Círculo)

Al presionar R1, el robot incrementa su velocidad de forma acelerada hasta alcanzar el valor máximo configurado y la mantiene durante un tiempo determinado, simulando una embestida directa contra el robot rival; al finalizar este intervalo se ejecuta automáticamente el frenado activo. El botón Círculo, por su parte, ejecuta un giro rápido de 180° sobre el propio eje del robot, útil como maniobra evasiva cuando el robot queda en una posición desfavorable cerca del borde del área de combate.

Mecanismo Anti-Flipper (DS3218MG)

El servomotor anti-flipper se controla mediante un canal LEDC configurado específicamente a 50 Hz con una resolución de 16 bits, lo que permite traducir un ángulo entre 0° y 180° en un ancho de pulso de entre 1000 y 2000 microsegundos, tal como lo requieren los servomotores estándar. Al presionar el botón Cuadrado se activa una secuencia automática y no bloqueante compuesta por tres fases: primero el servomotor se desplaza rápidamente hacia su posición de ataque; después permanece abierto durante un breve periodo de tiempo configurable, manteniendo la presión mecánica sobre el robot contrario; y finalmente regresa de forma gradual a su posición de reposo. Esta secuencia se actualiza en cada ciclo del programa sin detener el resto de las funciones del robot, e incluye un límite de seguridad de dos segundos que reinicia el mecanismo en caso de que la secuencia no logre completarse correctamente.

Ajuste de Velocidades

A partir de las pruebas realizadas con la primera configuración del control manual, se identificó que el robot resultaba demasiado lento para un combate de sumo competitivo. Por ello se

incrementó la velocidad base, la velocidad de turbo y la duración de la embestida respecto a los valores iniciales, manteniendo siempre activas las rampas de aceleración para evitar que el aumento de velocidad provocara un patinaje excesivo de las llantas.

Problemas Encontrados y Soluciones

Durante la transformación del robot al control manual mediante mando PS4 se identificaron y resolvieron varios problemas técnicos, agrupados en las siguientes etapas:

Etapas 1. Instalación de Librerías y Conexión Bluetooth

Problema: el IDE de Arduino no reconocía la librería PS4Controller ni lograba compilar el código relacionado con el control Bluetooth.

Causa: la librería no estaba instalada, y existían conflictos de compatibilidad entre la versión del núcleo (core) de ESP32 instalado y la versión de la librería utilizada.

Solución:

- Instalación manual de la librería PS4Controller.
- Verificación y ajuste de la versión del núcleo de ESP32 compatible con la librería.
- Inclusión de la cabecera esp_mac.h, necesaria para la obtención de direcciones MAC.

Problema: el mando no lograba conectarse de forma estable, o intentaba conectarse a dispositivos distintos de la ESP32.

Causa: la dirección MAC del mando no estaba configurada en el código, o el control conservaba un emparejamiento previo con otro dispositivo Bluetooth.

Solución:

- Obtención de la dirección MAC física del mando PS4.
- Configuración de dicha dirección directamente en el código mediante PS4.begin(MAC).
- Reinicio del módulo Bluetooth del mando antes de cada prueba.

Etapas 2. Funcionamiento del Servomotor Anti-Flipper

Problema: al presionar el botón asignado, el servomotor DS3218MG no respondía ni cambiaba de posición.

Causa: la configuración inicial del canal PWM utilizaba la frecuencia pensada para los motores (1 kHz) en lugar de los 50 Hz requeridos por un servomotor estándar, por lo que la señal enviada no era interpretada correctamente.

Solución:

- Configuración de un canal LEDC independiente y exclusivo para el servomotor, a 50 Hz y 16 bits de resolución.
- Verificación del cálculo de ancho de pulso (1000-2000 μ s) para todo el rango de 0° a 180°.

Problema: una vez que el servomotor comenzó a responder, presentaba vibraciones y movimientos bruscos no solicitados, especialmente cuando los motores de tracción funcionaban de forma simultánea.

Causa: caída de tensión y ruido eléctrico en la línea de alimentación compartida entre los motores y el servomotor, lo que interfería con la señal de control.

Solución:

- Separación de la alimentación del servomotor respecto a la de los motores de tracción.
- Verificación de la conexión a tierra común entre todos los componentes.
- Actualización gradual (rampa) del ángulo del servomotor, en lugar de movimientos bruscos instantáneos.

Etapas 3. Reacomodo Mecánico para la Instalación del Armazón

Problema: los componentes electrónicos (motores, controladores BTS7960, ESP32-CAM, batería y servomotor) ocupaban posiciones que impedían el montaje del armazón de protección sobre el chasis del robot.

Causa: la distribución física utilizada en la primera etapa del proyecto no contemplaba el espacio adicional requerido por el armazón ni por el mecanismo anti-flipper.

Solución:

- Reorganización de la posición de motores, controladores, batería y servomotor sobre el chasis.
- Reubicación y extensión del cableado para mantener todas las conexiones tras el movimiento de las piezas.
- Verificación del centro de gravedad del robot tras el reacomodo, para evitar inestabilidad durante los giros y las embestidas.

El problema más relevante de esta etapa fue la combinación entre la falta de margen eléctrico del sistema y la necesidad de un servomotor de alto torque para el mecanismo anti-flipper: al operar los motores de tracción y el servomotor de forma simultánea, la demanda de corriente superaba la capacidad de la fuente de alimentación original, lo que se manifestó primero como temblores en el servomotor y finalmente como una pérdida total de movimiento, hasta que se ajustó la distribución de la alimentación eléctrica del sistema.

Conclusiones

En conjunto, el proyecto demuestra que la evolución del robot sumo, de un esquema autónomo basado en visión artificial hacia un control manual mediante mando PS4, representó una mejora significativa en términos de velocidad, capacidad de respuesta y confiabilidad operativa frente a las exigencias de un combate real. Al eliminar la dependencia de la computadora externa, la cámara IP y la red WiFi, el sistema ganó autonomía energética y redujo los puntos de falla asociados a la latencia y a la estabilidad de la conexión inalámbrica, factores críticos en un enfrentamiento donde las decisiones se toman en fracciones de segundo.

El desarrollo también dejó en evidencia que el éxito de un sistema mecatrónico de este tipo no depende únicamente de la lógica de control, sino de la correcta integración entre software, electrónica y mecánica. El mezclador de joystick tipo arcade y las rampas de aceleración con frenado activo resolvieron el reto de traducir un único joystick en movimientos proporcionales y controlados, aprovechando el alto torque de los motores sin sacrificar la maniobrabilidad. De manera similar, la incorporación del servomotor como mecanismo anti-flipper amplió las capacidades tácticas del robot, aunque su implementación reveló que añadir actuadores de alta demanda eléctrica exige un dimensionamiento cuidadoso de la fuente de alimentación, ya que de lo contrario se generan fallas de comportamiento difíciles de diagnosticar a simple vista.

Asimismo, los inconvenientes surgidos durante la integración de componentes, tanto en la distribución física del armazón de protección como en la compatibilidad de la librería PS4Controller con el núcleo de ESP32, reforzaron la importancia de la planificación temprana en proyectos de robótica: anticipar el espacio mecánico necesario para futuros accesorios y verificar la compatibilidad de software y hardware antes de avanzar evita retrabajos y optimiza los tiempos de desarrollo.

En definitiva, los ajustes realizados sobre el control, la alimentación y la estructura del robot no solo solucionaron problemas puntuales, sino que en conjunto produjeron una plataforma más robusta y mejor adaptada a las condiciones reales de competencia, validando el cambio de paradigma de un control autónomo a uno manual como una decisión acertada para este contexto específico.